

詳細改修計画書：RowId 統一化と Entity 型制約追加

作成日: 2026-08-11
対象: Employee Context を中心とした Entity/RowId 架構整備
総所要時間: 約 6.5 時間
テスト数: 504 個（全テスト実行）

改修の全体像

変更内容サマリー

フェーズ	改修内容	ファイル数	テスト数	所要時間
Phase 1	SharedKernel.RowId を abstract 化	1	7	30分
Phase 2	Employee Context の 5つの RowId 型改修	5	112	1時間
Phase 3	Entity 型制約追加	2	27	30分
Phase 4	Employee Entity 再設計	4	52	1.5時間
Phase 5	Mapper/Repository 改修	4	19	1時間
Phase 6	Application層確認	3	24	30分
Phase 7	テスト実行・検証	-	504	1時間

🔧 Phase 1: SharedKernel.RowId の抽象化

目的

sealed → abstract に変更し、各 Context の RowId が継承可能にする

対象ファイル

- src/SharedKernel/ValueObjects/Identifiers/RowId.cs

改修内容

現在の実装:

```
public sealed class RowId : ValueObject, IEquatable<RowId>
{
    private readonly long _value;
    public static RowId From(long value) { ... }
    public static RowId New() { ... }
}
```

改修後:

```
public abstract class RowId : PrimitiveValueObject<long>, IEquatable<RowId>
{
    protected RowId(long value, bool isSet) : base(value, isSet) { }
    protected RowId(bool isSet) : base(isSet) { }

    public long Value => ValueField;
    public abstract override void Validate(long normalized);
    public override string ToString() => ValueField.ToString();
    // Equals/GetHashCode は基底クラスから継承
}
```

テスト方法

```
dotnet test tests/SharedKernel.Tests/ValueObjects/Identifiers/ --filter "RowId"
# 期待結果: 7 テスト成功
```

リスク評価

- **リスク:** SharedKernel.RowId の既存使用箇所への影響（Infrastructure 層の DB採番処理）
- **対策:** Phase 2 実行後にビルド確認

チェックポイント

- ☐ ビルド成功
 - ☐ RowIdTests (7個) 成功
-

Phase 2: Employee Context の 5つの RowId 型改修

目的

既存の RowId 型を RowId を継承する形にリファクタ

対象ファイル

グループA: 必須型 (3個)

- src/Contexts/Employee/Employee.Domain/ValueObjects/Employee/PersonRowId.cs
- src/Contexts/Employee/Employee.Domain/ValueObjects/Employee/DepartmentRowId.cs
- src/Contexts/Employee/Employee.Domain/ValueObjects/Employee/EmployeeRowId.cs

グループB: オプション型 (2個)

- src/Contexts/Employee/Employee.Domain/ValueObjects/Employee/ManagerEmployeeRowId.cs
- src/Contexts/Employee/Employee.Domain/ValueObjects/Employee/ParentDepartmentRowId.cs

改修パターン

グループA: 必須型の改修パターン

現在:

```
public sealed class PersonRowId : PrimitiveValueObject<long>, IEquatable<PersonRowId>
{
    public const long MinValue = 1L;
    public long Value => ValueField;

    private PersonRowId(long value) : base(value, true) { }

    public static PersonRowId From(long value) => new(value);
    public static bool TryFrom(long value, out PersonRowId result) { ... }
    public static bool TryFromDbValue(long value, out PersonRowId result) { ... }

    public override void Validate(long normalized)
    {
        if (normalized < MinValue)
            throw new ArgumentOutOfRangeException(...);
    }
}
```

改修後:

```

public sealed class PersonRowId : RowId, IEquatable<PersonRowId>
{
    public const long MinValue = 1L;

    private PersonRowId(long value) : base(value, true) { }

    public static PersonRowId From(long value) => new(value);
    public static bool TryFrom(long value, out PersonRowId result)
    {
        result = null!;
        try { result = From(value); return true; }
        catch (ArgumentOutOfRangeException) { return false; }
    }

    public static bool TryFromDbValue(long value, out PersonRowId result)
    {
        result = null!;
        try { result = From(value); return true; }
        catch (ArgumentOutOfRangeException) { return false; }
    }

    public override void Validate(long normalized)
    {
        if (normalized < MinValue)
            throw new ArgumentOutOfRangeException(...);
    }

    // IEquatable 実装は削除可能（基底クラスで十分）
    public bool Equals(PersonRowId? other) => ... // 残す場合
}

```

主な変更点:

1. `: PrimitiveValueObject<long>` → `: RowId`
2. `using` を `ValueObjects.Identifiers` に変更
3. `Abstractions` の `using` を削除
4. 他の メソッド/プロパティは基本的に同じ

グループB: オプション型の改修パターン

現在:

```

public sealed class ManagerEmployeeRowId : PrimitiveValueObject<long?>, IEquatable<ManagerEmployeeRowId>
{
    public static ManagerEmployeeRowId Unset() => new(null, false);
    public bool HasManager => IsSet;
}

```

```
// ... TryFrom で null 吸収ロジック ...  
}
```

改修後:

```
public sealed class ManagerEmployeeRowId : RowId, IEquatable<ManagerEmployeeRowId>  
{  
    // long? ではなく long を使用 (Unset 状態は IsSet=false で表現)  
  
    public static ManagerEmployeeRowId Unset() => new(false); // IsSet=false のコンストラクタ  
    public bool HasManager => IsSet;  
  
    public static bool TryFrom(long? input, out ManagerEmployeeRowId result)  
    {  
        result = null!;  
        if (!input.HasValue)  
        {  
            result = Unset();  
            return true;  
        }  
        try { result = new(input.Value); return true; }  
        catch { return false; }  
    }  
}
```

注意点: オプション型は long ではなく long? ではなく、IsSet フラグで Unset を表現

改修手順

1. PersonRowId を改修 (既に完了)

```
# 確認: grep "RowId" src/Contexts/.../PersonRowId.cs
```

2. DepartmentRowId を改修

- PrimitiveValueObject<long> → RowId に変更
- using を更新

3. EmployeeRowId を改修

- 同じパターン

4. ManagerEmployeeRowId を改修 (オプション型)

- PrimitiveValueObject<long?> → RowId に変更

- Unset() の実装を IsSet フラグベースに変更
- TryFrom で null 吸収ロジック調整

5. ParentDepartmentRowId を改修（オプション型）

- ManagerEmployeeRowId と同じパターン

テスト方法

```
# 各改修後に実行
dotnet test tests/Contexts/Employee.Domain.Tests/ValueObjects/Employee/ --filter "RowId"
# 期待結果: 112 テスト成功 (リグレッション 0)
```

リスク評価

- **リスク:** Validate メソッドが abstract になったため、すべての派生クラスが実装必須
- **対策:** コンパイラが強制。ビルド時に全実装をチェック

チェックポイント

- ☐ 5つの RowId ファイルすべてが PrimitiveValueObject から RowId 継承に変更
- ☐ 全RowIdTests (112個) 成功
- ☐ ビルド成功（コンパイラエラーなし）

Phase 3: Entity 型制約追加

目的

Entity のジェネリック型パラメータに ValueObject 制約を追加

対象ファイル

- src/SharedKernel/Entities/Abstractions/Entity.cs
- src/SharedKernel/Entities/Abstractions/AggregateRoot.cs

改修内容

Entity.cs:

```
// 現在
public abstract class Entity<TId> : IEquatable<Entity<TId>>
```

```
where TId : notnull
```

```
// 改修後
```

```
public abstract class Entity<TId> : IEquatable<Entity<TId>>  
    where TId : notnull, ValueObject
```

AggregateRoot.cs:

```
// 現在
```

```
public abstract class AggregateRoot<TId> : Entity<TId>  
    where TId : notnull
```

```
// 改修後
```

```
public abstract class AggregateRoot<TId> : Entity<TId>  
    where TId : notnull, ValueObject // 明示的に記載（可読性）
```

テスト方法

```
dotnet test tests/SharedKernel.Tests/Entities/  
# 期待結果: 27 テスト成功
```

リスク評価

- **リスク:** 低（既存すべての ID 型が ValueObject を継承しているため互換性あり）
- **対策:** ビルド時に型チェック

チェックポイント

- ☐ SharedKernel.Entities のビルド成功
- ☐ Architecture.Tests (27個) 成功
- ☐ Employee.Domain.Entities のビルド成功



Phase 4: Employee Entity 再設計

目的

Employee Entity を EmployeeRowId ベースに再構築

対象ファイル

主要改修

- `src/Contexts/Employee/Employee.Domain/Entities/Employee.cs`

副次改修（子Entity）

- `src/Contexts/Employee/Employee.Domain/Entities/DepartmentMembership.cs`
- `src/Contexts/Employee/Employee.Domain/Entities/RoleAssignment.cs`
- `src/Contexts/Employee/Employee.Domain/Entities/PermissionAssignment.cs`

改修内容

Employee.cs の全面改修

現在の構造:

```
public sealed class Employee : Entity<EmployeeId> // GUID ベース
{
    public EmployeeCode Code { get; }
    public PersonRowId PersonRowId { get; }
    // ... その他プロパティ ...
}
```

改修後の構造:

```
public sealed class Employee : AggregateRoot<EmployeeRowId> // long ベース
{
    // 同一性を表す ID
    public EmployeeRowId EntId { get; private set; } // Entity.Id と同期

    // ビジネス属性
    public EmployeeDivision Division { get; private set; }
    public EmployeeNumber Number { get; private set; }
    public EmployeeCode Code { get; private set; }
    public PersonRowId PersonRowId { get; private set; }

    // 監査フィールド (ValueObject)
    public CreatedAt CreatedAt { get; private set; }
    public CreatedBy CreatedBy { get; private set; }
    public UpdatedAt? UpdatedAt { get; private set; }
    public UpdatedBy? UpdatedBy { get; private set; }
    public DeletedAt? DeletedAt { get; private set; }
    public DeletedBy? DeletedBy { get; private set; }
}
```

```

// 退職情報（新規ValueObject）
public RetiredOn? RetiredOn { get; private set; }

// ドメインロジック
public bool IsActive(LocalDateTime asOf)
{
    if (!RetiredOn.HasRetired)
        return true;
    return asOf < RetiredOn.Value;
}

// ドメインイベント発行メソッド
protected void RetireEmployee(LocalDateTime retiredOn, IClock clock)
{
    RetiredOn = RetiredOn.FromValue(retiredOn);
    this.RaiseDomainEvent(new EmployeeRetiredEvent(
        this.EntId,
        retiredOn,
        clock.JstNow
    ));
}
}

```

新規 ValueObject: RetiredOn

実装場所: src/Contexts/Employee/Employee.Domain/ValueObjects/Employee/RetiredOn.cs

```

public sealed class RetiredOn : ValueObject, IEquatable<RetiredOn>
{
    private readonly LocalDateTime _value;

    private RetiredOn(LocalDateTime value, bool isSet)
    {
        _value = value;
        IsSet = isSet;
    }

    public LocalDateTime Value => _value;
    public bool HasRetired => IsSet;

    public static RetiredOn Unset() => new(LocalDateTime.MinValue, false);
    public static RetiredOn FromValue(LocalDateTime value) => new(value, true);

    public static bool TryFromDbValue(DateTime? dbValue, out RetiredOn result)
    {
        result = null!;
        if (!dbValue.HasValue)
        {
            result = Unset();
        }
    }
}

```

```

        return true;
    }
    try
    {
        var localDateTime = new LocalDateTime(dbValue.Value);
        result = FromValue(localDateTime);
        return true;
    }
    catch
    {
        return false;
    }
}

protected override IEnumerable<object?> GetValueComponents()
{
    yield return IsSet;
    yield return _value;
}

public bool Equals(RetiredOn? other) => ... // ValueObject 標準実装
public override bool Equals(object? obj) => Equals(obj as RetiredOn);
public override int GetHashCode() => GetHashCode.Combine(IsSet, _value);
public override string ToString() => IsSet ? _value.ToString() : "Not Retired";
}

```

子Entityの改修パターン

現在:

```

public sealed class DepartmentMembership : Entity<DepartmentMembershipId>
{
    public EmployeeRowId EmployeeRowId { get; }
    // ...
}

```

改修後:

```

public sealed class DepartmentMembership : Entity<EmployeeRowId>
{
    public EmployeeRowId EntId { get; private set; } // 親Entity との紐付け用
    // ...
}

```

改修手順

1. RetiredOn ValueObject を新規作成

- 実装: LocalDateTime をラップ、Unset 対応

2. Employee.cs を改修

- ID 型変更: `Entity<EmployeeId>` → `AggregateRoot<EmployeeRowId>`
- プロパティ追加: `EntId`, `Division`, `Number`
- プロパティ追加: `RetiredOn`
- メソッド追加: `IsActive()`, `RetireEmployee()`
- ドメインイベント: `EmployeeRetiredEvent` 発行

3. DepartmentMembership, RoleAssignment, PermissionAssignment を改修

- ID 型変更: `Entity<*Id>` → `Entity<EmployeeRowId>`
- `EntId` プロパティ追加

テスト方法

```
dotnet test tests/Contexts/Employee.Domain.Tests/Entities/  
# 期待結果: 52 テスト成功 (EmployeeTests は大幅更新必須)
```

リスク評価

- リスク: 高 (ID 型の大幅変更、既存テスト大幅書き換え必須)
- 対策: 段階的実装、各Entity ごとに テスト実行確認

チェックポイント

- ☐ RetiredOn ValueObject ビルド成功
- ☐ Employee.cs ビルド成功
- ☐ EmployeeTests が新しい構造に対応
- ☐ EmployeeTests (22個) 成功



Phase 5: Mapper/Repository 改修

目的

DbModel ↔ Entity の変換ロジックを EmployeeRowId ベースに更新

対象ファイル

Mapper

- `src/Contexts/Employee/Employee.Infrastructure/Mappers/EmployeeMapper.cs`
- `tests/Contexts/Employee.Infrastructure.Tests/Mappers/EmployeeMapperTests.cs`

Repository

- `src/Contexts/Employee/Employee.Infrastructure/Repositories/EmployeeRepository.cs`
- `tests/Contexts/Employee.Infrastructure.Tests/Repositories/EmployeeRepositoryTests.cs`

改修内容

EmployeeMapper の改修

現在:

```
public class EmployeeMapper
{
    public Employee ToDomainEntity(EmployeeDbModel dbModel, IClock clock)
    {
        var employeeId = EmployeeId.From(dbModel.EmployeeId);
        var employee = new Employee(employeeId, ...);
        return employee;
    }
}
```

改修後:

```
public class EmployeeMapper
{
    public Employee ToDomainEntity(EmployeeDbModel dbModel, IClock clock)
    {
        // DbModel の employee_row_id → EmployeeRowId に変換
        if (!EmployeeRowId.TryFromDbValue(dbModel.EmployeeRowId, out var rowId))
            throw new InvalidOperationException(...);

        // RetiredOn の DateTime → LocalDateTime 変換
        RetiredOn? retiredOn = null;
        if (dbModel.RetiredOn.HasValue)
        {
            RetiredOn.TryFromDbValue(dbModel.RetiredOn, out var ro);
            retiredOn = ro;
        }
        else
    }
```

```

    {
        retiredOn = RetiredOn.Unset();
    }

    // Entity の再構築
    var employee = new Employee(
        rowId: rowId,
        division: EmployeeDivision.From(dbModel.Division),
        number: EmployeeNumber.From(dbModel.EmployeeNumber),
        ...
        retiredOn: retiredOn
    );

    return employee;
}

public EmployeeDbModel ToDbModel(Employee entity)
{
    return new EmployeeDbModel
    {
        EmployeeRowId = entity.EntId.Value, // long 値
        Division = entity.Division.Value,
        EmployeeNumber = entity.Number.Value,
        RetiredOn = entity.RetiredOn?.Value.Value, // LocalDateTime → DateTime
        ...
    };
}
}

```

Repository の改修

変更内容:

- GetByIdAsync の ID パラメータを `EmployeeRowId` に変更
- EmployeeId (ビジネスID) での検索は別メソッド `GetByEmployeeNumberAsync` に分離

```

public class EmployeeRepository : IEmployeeRepository
{
    // PK 検索 (RowId ベース)
    public async Task<Employee?> GetByIdAsync(EmployeeRowId id)
    {
        var dbModel = await _dbContext.Employees
            .FirstOrDefaultAsync(e => e.EmployeeRowId == id.Value);
        return dbModel == null ? null : _mapper.ToDomainEntity(dbModel, _clock);
    }

    // ビジネスID 検索 (EmployeeNumber ベース)
    public async Task<Employee?> GetByEmployeeNumberAsync(EmployeeNumber number)
    {

```

```
var dbModel = await _dbContext.Employees
    .FirstOrDefaultAsync(e => e.EmployeeNumber == number.Value);
return dbModel == null ? null : _mapper.ToDomainEntity(dbModel, _clock);
}

public async Task<Employee> AddAsync(Employee employee)
{
    var dbModel = _mapper.ToDbModel(employee);
    await _dbContext.Employees.AddAsync(dbModel);
    await _dbContext.SaveChangesAsync();

    // INSERT 後、DB で採番された RowId を取得
    dbModel = await _dbContext.Employees
        .FirstAsync(e => e.EmployeeRowId == dbModel.EmployeeRowId);

    return _mapper.ToDomainEntity(dbModel, _clock);
}
}
```

テスト方法

```
dotnet test tests/Contexts/Employee.Infrastructure.Tests/
# 期待結果: 19 テスト成功 (EmployeeMapperTests, EmployeeRepositoryTests)
```

リスク評価

- **リスク:** 中 (DB マッピングロジックの変更。既存テストで新しい ID 型をカバーする必要)
- **対策:** DbModel テスト、Mapper テスト で十分にカバー

チェックポイント

- ☐ EmployeeMapper ビルド成功
- ☐ EmployeeRepository ビルド成功
- ☐ EmployeeMapperTests (10個) 成功
- ☐ EmployeeRepositoryTests (9個) 成功



Phase 6: Application層 Use Case の確認

目的

DTO と Use Case が新しい ID 体系に対応しているか確認

対象ファイル

- `src/Contexts/Employee/Employee.Application/Dtos/CreateEmployeeRequest.cs`
- `src/Contexts/Employee/Employee.Application/Dtos/UpdateEmployeeRequest.cs`
- `src/Contexts/Employee/Employee.Application/Dtos/EmployeeDto.cs`

確認項目

1. CreateEmployeeRequest

- どの ID を入力として受け取るか？（EmployeeRowId？PersonRowId？）
- レスポンスには何を返すか？

2. UpdateEmployeeRequest

- 更新対象の識別（EmployeeRowId で更新？）

3. EmployeeDto

- ビジネスID（EmployeeNumber）と RowId の両方を返す必要があるか？

テスト方法

```
dotnet test tests/Contexts/Employee.Application.Tests/  
# 期待結果: 24 テスト成功
```

チェックポイント

- ☐ Application.Tests (24個) 成功
- ☐ DTO マッピング ロジック が正しく動作

Phase 7: 全テスト実行・検証

目的

すべての改修が正しく実装され、リグレッションがないこと確認

テスト実行順序

```
# Step 1: ビルド確認  
dotnet build --no-incremental
```

```
# Step 2: 単体テスト（各フェーズの最小単位）
dotnet test tests/SharedKernel.Tests/ValueObjects/Identifiers/ --filter "RowId"
# 期待: 7 成功

dotnet test tests/Contexts/Employee.Domain.Tests/ValueObjects/Employee/
# 期待: 112 成功

dotnet test tests/SharedKernel.Tests/Entities/
# 期待: 27 成功

dotnet test tests/Contexts/Employee.Domain.Tests/Entities/
# 期待: 52 成功

dotnet test tests/Contexts/Employee.Infrastructure.Tests/
# 期待: 19 成功

dotnet test tests/Contexts/Employee.Application.Tests/
# 期待: 24 成功

# Step 3: Architecture テスト
dotnet test tests/SharedKernel.Tests/Architecture/
# 期待: 27 成功

# Step 4: 全テスト実行
dotnet test
# 期待: 504 成功（リグレッション 0）
```

リスク評価と対策

リスク	確度	対策
RowId 基底化で既存コード破損	低	Phase 1 直後 ビルド確認
Entity 互換性	低	Phase 3 直後 Architecture テスト
EmployeeId（GUID）→ EmployeeRowId（long）移行	中	Phase 5 単体テスト で十分検証
RetiredOn ValueObject 実装漏れ	低	コンパイラが型チェック
Mapper の DateTime ↔ LocalDateTime 変換エラー	中	Phase 5 EmployeeMapperTests で検証
Application層の DTO マッピング 不整合	低	Phase 6 Application.Tests で検証

チェックリスト

- ☐ Phase 1: RowId ビルド確認
- ☐ Phase 2: 5つの RowId ファイル ビルド + テスト 成功
- ☐ Phase 3: Entity 型制約 ビルド + Architecture テスト
- ☐ Phase 4: Employee Entity 再設計 ビルド + Domain テスト
- ☐ Phase 5: Mapper/Repository ビルド + Infrastructure テスト
- ☐ Phase 6: Application層 確認 ビルド + Application テスト
- ☐ Phase 7: 全テスト実行 504個 すべて 成功

ロールバック計画

万が一 Phase 中盤で重大な問題が発生した場合：

1. Phase 1 直後 ビルド失敗時

- RowId.cs を以前のバージョンに戻す
- （Phase 1 の改修は最小限なため、リスク低い）

2. Phase 2 テスト大量失敗時

- 5つの RowId ファイルを以前のバージョンに戻す
- RowId.cs に戻す

3. Phase 3 以降 の場合

- Entity.cs / AggregateRoot.cs を戻す
- 型制約を外す

4. Phase 4 大規模失敗時

- Employee.cs の改修を一度保存して、段階的に実装し直す

進捗トラッキング

実装中のチェック項目：

フェーズ	状態	完了条件	確認日時
Phase 1	 進行中	ビルド成功 + RowIdTests 成功	

フェーズ	状態	完了条件	確認日時
Phase 2	🕒 次	RowId 型 ビルド成功 + 112 テスト成功	
Phase 3	🕒 次	Entity テスト 成功 + Architecture テスト成功	
Phase 4	🕒 次	Employee Entity ビルド成功 + Domain テスト成功	
Phase 5	🕒 次	Mapper/Repository テスト成功	
Phase 6	🕒 次	Application テスト成功	
Phase 7	🕒 次	全テスト 504個 成功	

注記

既存の既知の問題

- EmployeeId (GUID) は将来的に廃止予定 (EmployeeNumber に統一)
- DomainEvent の AggregateRootId は EventId (GUID) に統一済み

今後の拡張

- AggregateRoot に子Entity管理機能を追加予定
- Repository pattern の非同期最適化
- Domain Event の Outbox パターン実装